PRODUCT REQUIREMENTS DOCUMENT (PRD)

EdgeErgo

A Privacy-Preserving Distributed Edge AI System for Ergonomic and Facial-Expression Monitoring

Document Version: 3.0
Project Stage: Implementation / Development / Deployment
Project Type: B.Tech Computer Science & Engineering Final-Year Project
Primary Domains: Artificial Intelligence, Computer Vision, Edge Computing, Occupational Ergonomics, Distributed Systems

---

1. Executive Summary

EdgeErgo is a privacy-preserving, distributed Edge AI system designed to monitor ergonomic posture and broad facial-expression states of workers operating computer-based workstations.

Unlike a centralized computer-vision system, EdgeErgo performs analysis independently on each worker's computer.

Each worker computer acts as an **Edge Node** containing a lightweight Edge AI Agent. The Edge Agent captures camera frames locally, performs pose and facial analysis, calculates ergonomic measurements, evaluates risk, performs temporal analysis, and provides immediate feedback to the worker.

Raw camera video is processed locally and is not transmitted to the central server.

Only summarized, anonymized and structured assessment results are transmitted to a common Central Server.

The Central Server aggregates data from multiple Edge Nodes and provides collective ergonomic analysis, workstation-level analysis, trends, CSV exports, and detailed reports.

The system therefore provides two levels of functionality:

Individual level

Camera
↓
Local Edge AI
↓
Ergonomic + Facial Analysis
↓
Worker Feedback Overlay

Collective level

Multiple Edge Nodes
↓
Anonymized Summaries
↓
Central Server
↓
Aggregation
↓
Collective Analysis
↓
Dashboard / CSV / Reports

The central design principle of EdgeErgo is:

> **Analyze locally. Alert locally. Store locally. Transmit only necessary summaries. Aggregate centrally.**

---

2. Product Vision

The vision of EdgeErgo is to provide an affordable, lightweight and privacy-preserving distributed AI system that can assist individual workers in maintaining better working posture while allowing organizations or supervisors to understand ergonomic trends across multiple workstations.

The system should transform:

Camera Input
↓

Local Edge Processing
↓
Body + Face Features
↓
Ergonomic / Expression Analysis
↓
Temporal Analysis
↓
Local Worker Feedback
↓
Local Summary
↓
Anonymized Transmission
↓
Central Aggregation
↓
Collective Analysis
↓
Dashboard + CSV + Report

The system must not depend on sending raw camera data to a cloud service.

---

3. Problem Statement

Workers who spend extended periods at computer workstations may experience ergonomic risks caused by poor sitting posture, forward neck posture, excessive trunk bending, inappropriate shoulder or arm positioning, repetitive activity, prolonged static posture, insufficient movement breaks, and other workstation-related behaviors.

Traditional ergonomic assessment generally depends on manual observation. This can be time-consuming, difficult to perform continuously, and unsuitable for providing immediate feedback.

A centralized computer-vision approach introduces an additional privacy concern because camera data from multiple workers would need to be transmitted to a central processing system.

EdgeErgo addresses these problems through a distributed architecture in which each worker's computer performs its own computer-vision processing.

The central system receives only the results required for collective analysis.

---

4. Product Goals

4.1 Primary Goals

1. Perform computer-vision analysis locally on worker computers.
2. Detect human body landmarks using lightweight pose estimation.
3. Calculate relevant posture and joint measurements.
4. Perform ergonomic assessment using RULA and/or REBA.
5. Analyze broad facial-expression states locally.
6. Perform analysis according to configurable time intervals.
7. Detect persistent posture conditions rather than reacting to a single frame.
8. Provide immediate feedback to the worker through a desktop overlay.
9. Provide movement-break and hydration reminders.
10. Store assessment results locally on each Edge Node.
11. Continue local operation when the central server is unavailable.
12. Synchronize unsent summaries when connectivity is restored.
13. Send only anonymized structured results to the Central Server.
14. Aggregate data from multiple Edge Nodes.
15. Generate collective ergonomic analysis.
16. Generate collective facial-expression statistics.
17. Generate combined trend analysis.
18. Provide a central dashboard.
19. Provide CSV export.
20. Generate detailed reports.
21. Minimize CPU, memory and network usage.
22. Provide a practical deployable application that can be installed and tested on different computers.

---

5. Secondary Goals

The system should also:

- Detect when an Edge Node is online or offline.
- Track Edge Agent software versions.
- Provide camera and configuration diagnostics.
- Handle temporary network failures.
- Handle temporary pose-detection failures.
- Avoid repeated unnecessary alerts.
- Support multiple Edge Nodes communicating with one Central Server.

- Provide a simple installation process.
- Minimize technical interaction required from the worker.

---

6. Non-Goals

The initial system will NOT attempt to:

- Diagnose medical conditions.
- Predict or confirm individual injuries.
- Diagnose depression, anxiety, stress, burnout or other psychological conditions.
- Perform facial recognition.
- Identify workers using facial identity.
- Store raw worker video on the Central Server.
- Continuously upload camera frames.
- Replace a certified ergonomist or medical professional.
- Determine a person's psychological state from facial expression.
- Perform unrestricted multi-person ergonomic assessment.
- Build a large enterprise workforce-surveillance platform.
- Require cloud-based AI processing for core functionality.
- Use large computationally expensive models unnecessarily.

---

7. Target Users

7.1 Worker / Individual

A person operating a computer workstation who receives local posture and activity feedback.

Examples:

- Office workers
- Software developers
- Students
- Laboratory users
- Computer laboratory users
- Work-from-home users
- Computer-based industrial workers

7.2 Supervisor / Administrator

A person who reviews aggregated ergonomic statistics across multiple workstations.

The administrator should primarily see analytical results rather than raw camera data.

---

8. Core Product Concept

EdgeErgo consists of two major layers.

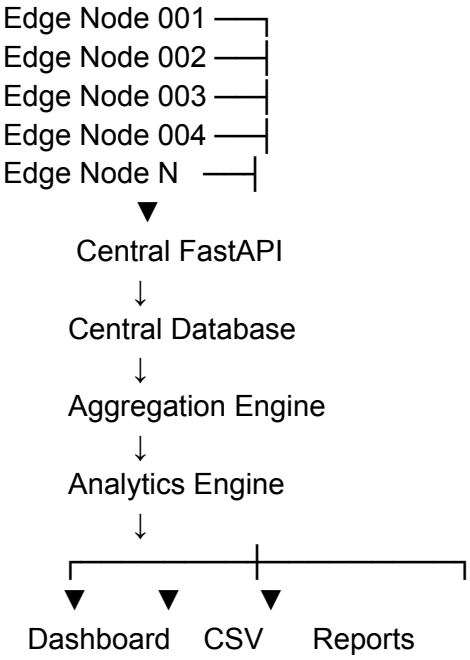
Layer 1 — Edge Layer

Every worker computer runs an independent EdgeErgo Edge Agent.

Camera
↓
OpenCV
↓
Preprocessing
↓
Pose Detection
↓
Face Detection / Facial Features
↓
Feature Extraction
↓
Ergonomic Engine
↓
Expression Analysis
↓
Temporal Analysis
↓
Feedback Engine
↓
Local SQLite
↓
Synchronization

Layer 2 — Central Layer

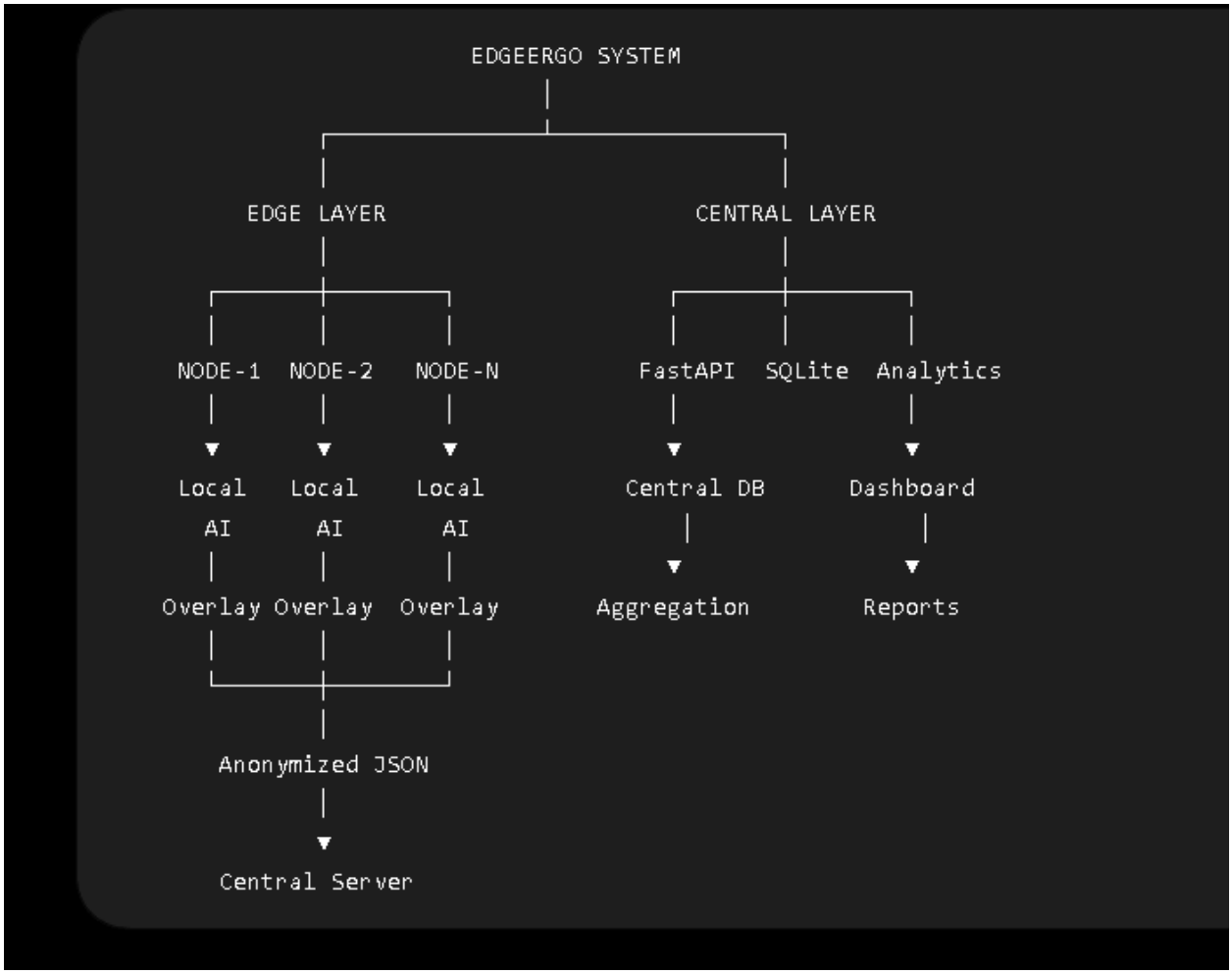
A single Central Server receives summaries from multiple Edge Nodes.



---

9. System Architecture

The overall system architecture shall be:



---

10. Edge Node Architecture

Every worker computer shall run an independent Edge Agent.

Edge Agent Responsibilities

The Edge Agent shall:

- Initialize the camera.
- Capture frames.
- Resize and preprocess frames.
- Detect body landmarks.
- Detect facial landmarks/features.

- Calculate body angles.
- Evaluate ergonomic posture.
- Estimate broad facial-expression states.
- Maintain temporal state.
- Detect persistent ergonomic issues.
- Generate worker feedback.
- Manage break reminders.
- Store results locally.
- Maintain synchronization status.
- Communicate with the Central Server.
- Send periodic health/heartbeat information.
- Continue functioning when the server is unavailable.

---

11. Central Server Architecture

The Central Server shall not perform raw camera analysis.

Its primary responsibilities are:

- Edge Node registration.
- Device identification.
- Authentication/validation.
- Data ingestion.
- Health monitoring.
- Data storage.
- Aggregation.
- Collective analysis.
- Dashboard API.
- CSV generation.
- Report generation.
- Trend analysis.

The server shall receive summarized metrics instead of video frames.

---

12. Recommended Technology Stack

12.1 Worker / Edge Computer

Python

Primary programming language.

OpenCV

Responsibilities:

- Camera access
- Frame acquisition
- Frame preprocessing
- Resizing
- Image operations
- Local visualization

MediaPipe Pose

Responsibilities:

- Body landmark extraction
- Skeleton information
- Pose visibility/confidence

MediaPipe Face / Lightweight Face Processing

Responsibilities:

- Face detection
- Facial landmark extraction
- Feature calculation
- Broad expression classification

NumPy

Responsibilities:

- Vector calculations
- Coordinate processing
- Mathematical operations
- Angle calculations

Python RULA/REBA Engine

Responsibilities:

- Ergonomic scoring
- Rule evaluation
- Risk classification

SQLite

Responsibilities:

- Local session data
- Local assessment summaries
- Event storage
- Unsynchronized data
- Sync queue

PySide6

Responsibilities:

- Desktop overlay
- Worker notification window
- Status display
- Basic configuration interface

---

13. Central Server Technology Stack

Python

Main server language.

FastAPI

Responsibilities:

- Edge API
- Device registration
- Data ingestion
- Health monitoring
- Dashboard API
- Report API

SQLite

Used initially as the Central Server database.

SQLite is appropriate for the initial controlled prototype because it is lightweight, requires no separate database server and is easy to deploy.

The architecture should keep the database layer modular so that PostgreSQL can be introduced later without rewriting the entire application.

React + Vite

Used for the **Central Administrator Dashboard only**.

React shall NOT be required on worker computers.

---

14. Final Technology Architecture

WORKER COMPUTER

Python Edge Agent

OpenCV
↓
MediaPipe Pose
↓
MediaPipe Face
↓
NumPy
↓
RULA / REBA
↓
Expression Analysis
↓
Temporal Engine
↓
Feedback Engine
↓
PySide6 Overlay
↓

SQLite
↓
JSON Summary

CENTRAL COMPUTER
———————————————————

FastAPI
↓
Central SQLite
↓
Aggregation Engine
↓
Analytics Engine
↓
React Dashboard
↓
CSV / Reports

---

15. Monitoring Schedule

EdgeErgo shall support scheduled analysis.

The system shall distinguish between:

Analysis Interval

How frequently an analysis session begins.

Example:

Every 30 minutes

Analysis Duration

How long each analysis session runs.

Example:

5 minutes

Example workflow:

09:00
↓
5-minute analysis
↓
09:05
↓
Idle
↓
09:30
↓
5-minute analysis
↓
09:35
↓
Idle

This design reduces computational load compared with continuously running full inference.

---

16. Monitoring Configuration

The user or administrator may configure:

- Monitoring start time
- Monitoring end time
- Analysis interval
- Analysis duration
- Posture warning duration
- Break reminder interval
- Alert cooldown
- Camera selection
- Overlay preferences

Example:

Monitoring:
Start: 09:00
End: 17:00

Analysis:
Interval: 30 minutes
Duration: 5 minutes

Posture Warning:
Persistence: 20 seconds

Break Reminder:
Interval: 60 minutes

Actual default values shall be finalized after performance testing.

---

17. Computational Optimization

The system shall be designed to minimize unnecessary computation.

17.1 Reduced Camera Resolution

The application should use a practical low-resolution camera feed where sufficient for accurate landmark detection.

Example:

640 × 480

The final resolution shall be determined through benchmarking.

17.2 Frame Skipping

Camera capture may occur at a higher frame rate than AI inference.

Example:

Camera:
30 FPS

AI inference:
5–10 FPS

Only selected frames are processed.

17.3 Scheduled Analysis

Full analysis is performed only during the configured analysis windows.

17.4 Temporal Aggregation

Individual frame results should be aggregated locally rather than continuously transmitted.

17.5 Event-Based Feedback

Feedback shall only be triggered when a condition is sufficiently persistent.

---

18. Camera Processing

The Edge Agent shall perform:

Camera
↓
Frame Capture
↓
Resize
↓
Color Conversion
↓
Optional Preprocessing
↓
Pose / Face Processing

Frames shall normally remain in memory and be discarded after processing.

---

19. Pose Detection

The system shall use a lightweight pose-estimation solution to obtain required body landmarks.

Required landmark categories may include:

- Head / face region
- Shoulders
- Elbows
- Wrists
- Hips
- Knees
- Ankles

The Edge Agent shall also use visibility/confidence information where available.

Low-confidence landmarks should not immediately trigger high-risk warnings.

---

20. Skeleton Visualization

The Edge Agent may optionally display the detected skeleton.

The purpose is to:

- Visualize detected posture.
- Help debug the system.
- Help validate landmark positions.
- Help demonstrate the technology during project testing.

Skeleton visualization is optional during normal worker operation.

---

21. Geometry Engine

The geometry engine shall convert landmarks into relevant posture measurements.

Potential measurements include:

- Neck angle
- Trunk angle
- Shoulder angle
- Elbow angle
- Wrist angle
- Hip angle

- Knee angle

The geometry module shall be independent from the ergonomic scoring module.

Architecture:

Landmarks
↓
Coordinate Processing
↓
Vector Calculation
↓
Joint Angles
↓
Posture Parameters

---

22. Ergonomic Assessment Engine

The Ergonomic Engine shall accept posture measurements and produce ergonomic scores.

Body Landmarks
↓
Angles
↓
Posture Parameters
↓
RULA / REBA Rules
↓
Score
↓
Risk Category

The ergonomic scoring logic must follow the chosen assessment methodology rather than arbitrary angle-to-score mappings.

---

23. RULA

RULA shall be used primarily for upper-limb-related ergonomic assessment.

The system may evaluate:

- Upper arm
- Lower arm
- Wrist
- Neck
- Trunk
- Leg position
- Relevant modifiers

The implementation shall maintain the scoring tables separately from computer-vision logic.

---

24. REBA

REBA shall be used for whole-body ergonomic assessment.

The system may evaluate:

- Trunk
- Neck
- Legs
- Upper arm
- Lower arm
- Wrist
- Relevant activity factors
- Relevant load/force factors
- Coupling/activity factors where applicable

The scoring engine shall remain configurable.

---

25. Risk Classification

The system shall convert ergonomic scores into understandable risk categories.

Possible categories:

LOW
MODERATE
HIGH

VERY HIGH

The actual mapping shall be based on the selected ergonomic assessment standard.

The system shall not create arbitrary risk thresholds unrelated to the implemented standard.

---

26. Temporal Posture Analysis

A single incorrect frame should not automatically result in a warning.

The system shall monitor persistence.

Example:

Incorrect posture
↓
Persistence timer
↓
Condition continues?
↓
YES
↓
Trigger warning

Example:

Incorrect neck posture
↓
20 seconds
↓
Warning generated

The final persistence threshold shall be configurable.

---

27. Facial Analysis

A facial-analysis component shall be included as part of the Edge Agent.

Its purpose is to identify broad visible facial-expression states rather than personal identity or medical/psychological conditions.

Possible categories may include:

- Neutral
- Positive expression
- Negative expression
- Reduced-expression state
- Eye-closure-related event

The exact categories shall be finalized based on the available lightweight model and validation results.

The system shall NOT claim that an expression proves:

- Stress
- Depression
- Anxiety
- Burnout
- Mental illness
- Medical conditions

Facial analysis is intended only as an observational feature for aggregate analytics.

---

28. Facial Privacy

The following must remain local:

- Face images
- Face crops
- Facial landmark images
- Identity information
- Face embeddings

Only summarized expression metrics may be transmitted.

Example:


```
{
   "neutral_percentage": 72,
   "positive_percentage": 15,
   "negative_percentage": 8,
```

```
    "eye_closure_events": 2
}
```

---

29. Combined Ergonomic and Facial Analysis

The Central Server may examine relationships between:

- Ergonomic risk
- Facial-expression distributions
- Working duration
- Time of day
- Break events

Example:

```
Working Duration
      +
Ergonomic Risk
      +
Expression Distribution
      ↓
Observed Collective Pattern
```

The system should report such findings as **observed relationships or correlations**, not proof of causation.

---

30. Worker Feedback Overlay

A key feature of EdgeErgo is the local worker overlay.

The worker should receive feedback directly on the same computer being monitored.

Example:



The overlay shall:

- Be lightweight.
- Remain above normal applications where appropriate.
- Avoid blocking the user's work.
- Automatically disappear.
- Provide concise actionable guidance.
- Display only when needed.

---

31. Worker Awareness

The worker shall be able to see that EdgeErgo is active.

Example:

EdgeErgo ● Monitoring

The local application may provide:

- Monitoring status
- Current posture status
- Current risk status
- Last analysis time
- Next analysis time
- Recent warning
- Synchronization status

The worker should not need to open the administrator dashboard.

---

32. Posture Feedback

Examples:

Maintain a more neutral neck position.

Reduce forward trunk bending.

Adjust your shoulder position.

Maintain a more neutral arm position.

Consider changing your posture.

Recommendations shall be tied to detected conditions.

---

33. Break Reminders

The Edge Agent shall support activity-break reminders.

Example:

🚶 Movement Break

You've been working continuously
for a prolonged period.

Consider taking a short movement break.

The reminder timing shall be configurable.

---

34. Hydration Reminder

The system may provide general hydration reminders.

Example:

💧 Hydration Reminder

Consider taking a moment to drink some water.

This feature shall be implemented as a general reminder rather than a medical recommendation.

---

35. Alert Priority

Alerts shall have priority levels.

Example:

HIGH-RISK POSTURE
↓
POSTURE WARNING
↓
MOVEMENT BREAK
↓
HYDRATION REMINDER

Multiple notifications should not appear simultaneously.

---

36. Alert Cooldown

The same warning should not continuously repeat.

Example:

Warning
↓
Cooldown
↓
Condition checked again
↓
Still present?
↓
Warning may be shown again

This reduces notification fatigue.

---

37. Local Database

Each Edge Node shall maintain a local SQLite database.

The local database shall contain:

device
session
assessment
events
sync_queue
configuration

---

38. Local Data Model

Device

device_id
device_name
software_version
registered_at
last_sync

Session

session_id
device_id
start_time
end_time
duration

Assessment

assessment_id
session_id
timestamp
rula_score
reba_score

risk_level
dominant_posture
expression_category

Events

event_id
session_id
timestamp
event_type
event_value

Sync Queue

sync_id
record_id
created_at
status
retry_count
last_attempt

---

39. Local Offline Operation

If the Central Server becomes unavailable:

Edge Agent
↓
Local Analysis
↓
SQLite
↓
Sync Queue

The worker shall continue receiving local feedback.

The Edge Agent shall not stop ergonomic processing simply because the server is unavailable.

---

40. Synchronization

When the Central Server becomes available again:

Unsynchronized Records
↓
Connection Test
↓
Send Summary
↓
Server Validation
↓
Success
↓
Mark Synced

Failed uploads shall remain in the local queue.

The system shall retry later.

---

41. Central Communication

The Edge Agent shall transmit structured data using HTTP/JSON.

Example:

POST /api/v1/edge/assessment

The Edge Agent shall transmit summarized results rather than video frames.

---

42. Edge Node Registration

Every installation shall receive a unique anonymous Edge Node ID.

Examples:

NODE-001
NODE-002
NODE-003
NODE-004

The device ID should not depend on the worker's personal name.

The initial installation process should support automatic or administrator-approved registration.

---

43. First-Time Edge Setup

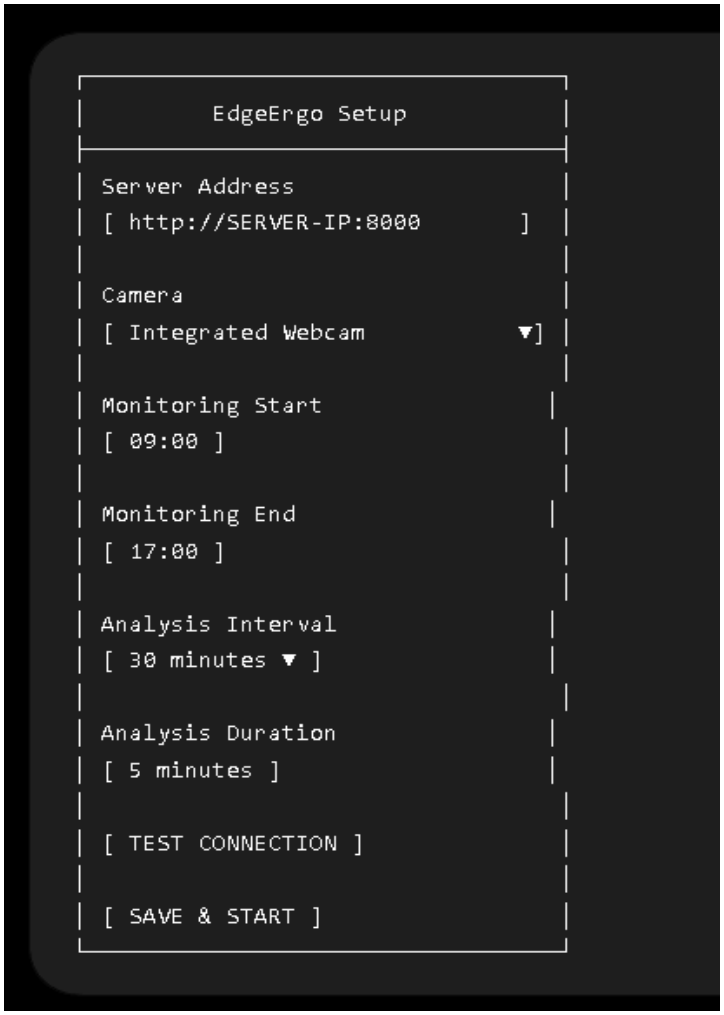
When EdgeErgo is installed on a new computer:

Installation
↓
First Launch
↓
Camera Detection
↓
Central Server Address
↓
Test Connection
↓
Node Registration
↓
Node ID Assigned
↓
Configuration Saved
↓
Edge Agent Ready

---

44. Edge Configuration Screen

The first-time setup may include:



---

45. Edge Agent Background Operation

After installation, the Edge Agent should be capable of operating in the background.

Expected flow:

Windows starts
↓
EdgeErgo Agent starts
↓

Agent initializes
↓
Camera available?
↓
Wait for scheduled analysis
↓
Analyze
↓
Store summary
↓
Synchronize

The implementation may support starting with the operating system after validation.

---

46. Edge Agent Health Monitoring

Each Edge Node should periodically send a lightweight heartbeat.

Example:

NODE-001 → ONLINE
NODE-002 → ONLINE
NODE-003 → ONLINE
NODE-004 → OFFLINE

The Central Server shall record:

- Last heartbeat
- Software version
- Node status
- Last telemetry window

The Central Dashboard shall show node availability.

---

47. Edge Agent Versioning

Every Edge Agent installation shall report its software version.

Example:

This allows the administrator to determine which nodes are running different versions.

Automatic software updating is optional for the MVP.

---

48. Central Data Storage

The Central Server shall store data received from multiple Edge Nodes.

The Central database may contain:

devices
sessions
telemetry_windows
posture_metrics
expression_metrics
events
reports

The Central database should store summaries rather than camera content.

---

49. Example Edge Telemetry Summary

A typical telemetry window may look like:

{
  "device_id": "NODE-004",
  "session_id": "S-20260914-001",
  "window_start": "2026-09-14T10:00:00",
  "window_end": "2026-09-14T10:05:00",

  "ergonomics": {
    "average_rula": 4.2,
    "maximum_rula": 6,
    "average_reba": 5.1,
    "maximum_reba": 7,

```
    "high_risk_percentage": 32.5,
    "dominant_posture": "NECK_FLEXION"
  },

  "facial_expression": {
    "neutral_percentage": 72,
    "positive_percentage": 15,
    "negative_percentage": 8,
    "reduced_expression_percentage": 5
  },

  "breaks": {
    "breaks_recommended": 1,
    "breaks_acknowledged": 0
  }
}
```


This is a representative structure; actual fields may evolve during implementation.

---

50. Collective Analysis

The Central Server shall combine telemetry from all Edge Nodes.

The system should be capable of calculating:

- Average RULA
- Maximum RULA
- Average REBA
- Maximum REBA
- Risk distribution
- High-risk percentage
- Most common posture defect
- Workstation-level risk
- Time-based risk
- Expression distribution
- Break frequency
- Break response
- Aggregate trends

---

51. Workstation Analysis

Each anonymous Edge Node may have its own summary.

Example:

NODE-004

Average RULA: 4.2
Average REBA: 5.1
High-Risk Percentage: 32%

Dominant Posture:
Neck Flexion

Primary Expression Category:
Neutral

The dashboard shall avoid displaying unnecessary personal identity information.

---

52. Collective Ergonomic Dashboard

The administrator dashboard shall provide:

Overview

Active Workstations: 18

Low Risk: 10
Moderate Risk: 6
High Risk: 2

Ergonomic Metrics

- Average RULA
- Average REBA
- Maximum observed score
- High-risk ratio
- Risk distribution
- Dominant posture defects
- Risk by time

- Risk by workstation

---

53. Facial-Expression Dashboard

The dashboard may display aggregate expression distributions.

Example:

Neutral: 72%
Positive: 15%
Negative: 8%
Reduced Expression: 5%

Results must be presented as observed classification statistics rather than psychological conclusions.

---

54. Combined Dashboard

The system may display:

Ergonomic Risk vs Expression Distribution

Time
↓
Risk
↓
Expression
↓
Observed Pattern

Possible analyses:

- Ergonomic risk vs expression category
- Risk vs work duration
- Risk vs break events
- Expression distribution vs time
- High-risk periods vs expression distribution

No causal claims shall be generated automatically.

---

55. Time-Based Analysis

The dashboard shall support time-based analysis.

Example:

09:00 → Low
10:00 → Moderate
11:00 → Moderate
12:00 → Low
14:00 → High
15:00 → High

This can help identify periods when ergonomic risk tends to increase.

---

56. Break Analysis

The Central Server shall aggregate:

- Number of break recommendations
- Number of acknowledged breaks
- Number of ignored reminders
- Break frequency
- Risk before and after breaks where sufficient data exists

The system shall not make medical claims based on break behavior.

---

57. CSV Export

The administrator shall be able to export aggregated data as CSV.

Example:

timestamp,
device_id,

session_id,
duration,
avg_rula,
max_rula,
avg_reba,
max_reba,
risk_level,
dominant_posture,
expression_category,
break_required

The actual CSV schema may evolve during implementation.

---

58. Report Generation

EdgeErgo shall generate reports for selected periods.

Supported periods may include:

- Session
- Day
- Week
- Month

The report shall contain:

1. Executive Summary

- Number of workstations
- Number of telemetry windows
- Monitoring duration
- Average ergonomic scores
- High-risk events

2. Ergonomic Findings

- Average risk
- Maximum risk
- Dominant posture defects
- High-risk periods

- Workstation trends

3. Facial-Expression Statistics

- Expression distribution
- Expression trends
- Time-based distribution

4. Combined Observations

- Observed relationships between posture and expression categories
- Risk vs duration
- Risk vs break activity

5. Recommendations

Examples:

Encourage regular movement breaks.

Review workstation monitor positioning.

Provide guidance on neutral neck posture.

Encourage appropriate workstation setup.

Review periods with consistently elevated
ergonomic risk.

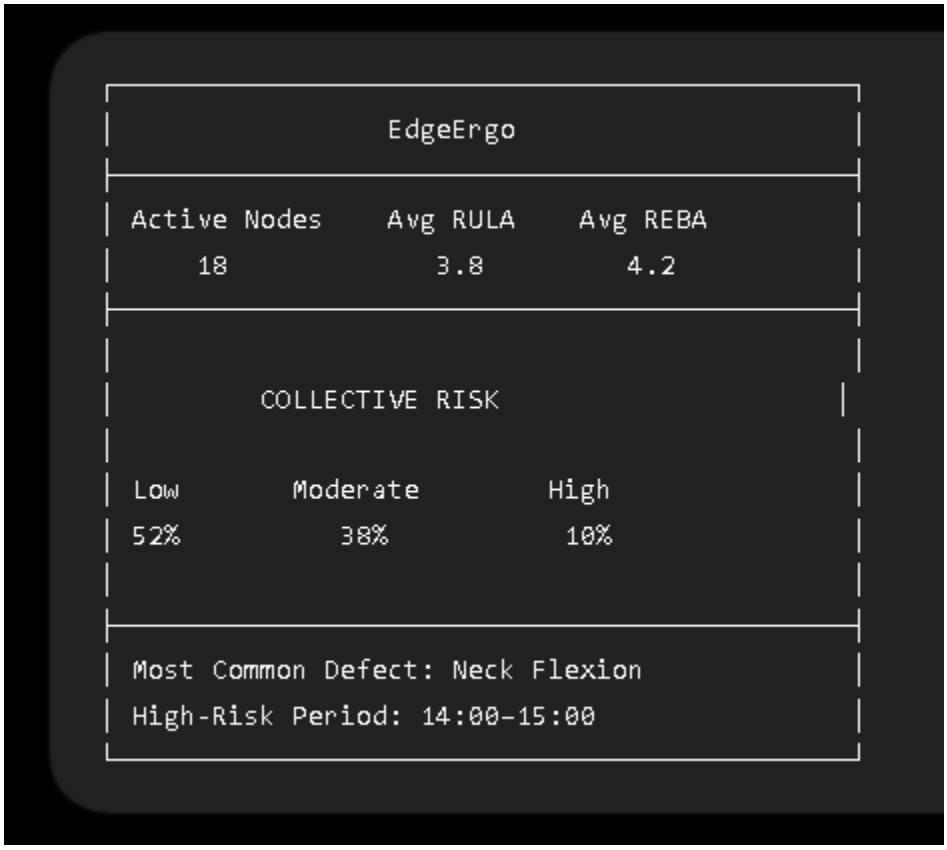
6. Privacy Summary

The report should state that raw camera data was processed locally and that only summarized analytical information was transmitted.

---

59. Central Dashboard Layout

The Central Dashboard may contain:



---

60. Workstation Table

The Central Dashboard should support a workstation table.

Example:

WORKSTATION NODE	STATUS	AVG RULA	AVG REBA	RISK	HIGH-RISK %	DEFECT
NODE-001	ONLINE	5.4	6.4	HIGH	88.9%	Neck Flexion
NODE-002	ONLINE	4.3	4.3	MOD	40.7%	Wrist Deviation
NODE-003	ONLINE	2.3	2.4	LOW	0.0%	None

The exact interface design may evolve.

---

61. Frontend Architecture

The system shall have two separate user-interface concepts.

Worker UI

Implemented using:

Python
+
PySide6

The worker does NOT need:

- React
- Node.js
- Vite
- Browser dashboard

for normal Edge Agent operation.

Administrator UI

Implemented using:

React
+
Vite

and served through the Central Server.

---

62. Security and Privacy Architecture

The system shall follow the principle:

RAW VIDEO
↓
LOCAL EDGE PROCESSING

↓
NUMERICAL / CATEGORICAL METRICS
↓
ANONYMIZED SUMMARY
↓
CENTRAL SERVER

The server shall not receive:

- Raw video
- Webcam frames
- Face photographs
- Screenshots
- Audio recordings
- Facial identity embeddings

unless explicitly added as a future feature and separately approved.

---

63. Data Minimization

The Edge Agent should transmit only data necessary for collective analysis.

Instead of:

30 camera frames/second

the system should transmit:

One aggregated summary per telemetry window

This reduces:

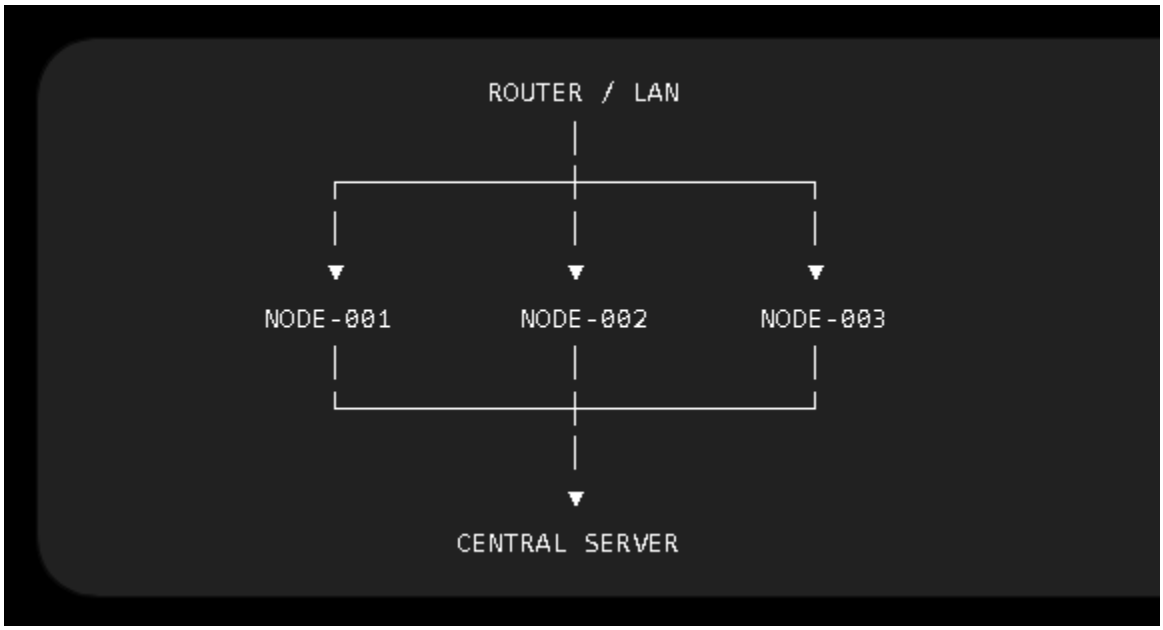
- Network traffic
- Central storage requirements
- Privacy exposure
- Server computational load

---

64. Network Architecture

For an initial deployment, the system may operate on a local network.

Example:



The Central Server can initially use an internal LAN address.

Secure transport should be introduced when deploying outside a trusted local network.

---

65. Server Failure Handling

If the server fails:

Edge Nodes
↓
Continue local processing
↓
Store locally
↓
Queue summaries

When the server returns:

Server restored

↓
Queued summaries uploaded
↓
Database updated

No local analysis should depend on server availability.

---

66. Camera Failure Handling

If the camera cannot be accessed:

Camera unavailable.

Please check camera connection
and permissions.

The application should not crash.

---

67. Person Not Detected

If no suitable pose is detected:

No suitable human pose detected.

The system should avoid generating false ergonomic results.

---

68. Low Confidence

If body or facial landmarks fall below an acceptable confidence threshold:

Pose visibility is insufficient.

Please adjust your position
or camera placement.

The corresponding sample should not automatically contribute a high-risk result.

---

69. Multiple-Person Condition

The initial system is designed for:

One workstation
↓
One primary worker

If multiple persons are detected in the camera view, the Edge Agent should avoid analyzing an ambiguous subject.

Possible message:

Multiple people detected.

Please ensure the workstation
user is clearly visible.

---

70. Edge Agent Deployment

The Edge Agent should be distributable as a normal application rather than requiring a developer environment.

The target deployment should resemble:

EdgeErgoSetup.exe
↓
Install
↓
EdgeErgo Agent
↓
Configuration
↓
Ready

Workers should not need to manually install:

- Python

- OpenCV
- MediaPipe
- NumPy
- PySide6

individually.

---

71. Edge Packaging

The Edge Agent should be packaged into an executable for the target operating system.

For Windows, a packaging solution such as PyInstaller may be used.

The packaged application should include required runtime dependencies.

Example:

```
EdgeErgo/
├── EdgeErgoAgent.exe
├── models/
├── config/
├── logs/
└── data/
```

---

72. Central Server Deployment

The Central Server may initially be deployed on:

- A dedicated PC
- A laptop
- A local workstation
- A controlled laboratory computer

The initial deployment does not require cloud infrastructure.

---

73. Central Server Packaging

The final demonstration should allow the Central Server to be started without requiring a development workflow.

The React frontend shall be built into production assets.

The Central Server may serve the built dashboard.

Conceptually:

Central Server
↓
FastAPI
↓
React Build
↓
Browser

The administrator can access the system through:

http://SERVER-IP

within the configured network.

---

74. Software Configuration

Edge configuration should contain items such as:

```
{
   "device_id": "NODE-001",
   "server_url": "http://192.168.1.10:8000",
   "camera_index": 0,
   "monitoring_start": "09:00",
   "monitoring_end": "17:00",
   "analysis_interval_minutes": 30,
   "analysis_duration_minutes": 5
}
```

Sensitive credentials, where applicable, should not be hardcoded.

---

75. Health Monitoring

The Central Dashboard should provide:

NODE-001 ● ONLINE
NODE-002 ● ONLINE
NODE-003 ● OFFLINE
NODE-004 ● ONLINE

The dashboard may also display:

- Last heartbeat
- Last telemetry
- Software version
- Sync state

---

76. Production Readiness Requirements

The Edge Agent shall be considered deployment-ready only when it can:

1. Install on a clean computer.
2. Detect the camera.
3. Register a node.
4. Save configuration.
5. Run without a developer environment.
6. Perform local inference.
7. Show the worker overlay.
8. Store results locally.
9. Connect to the Central Server.
10. Continue running while the server is unavailable.
11. Synchronize queued data after reconnection.
12. Report software version.
13. Report heartbeat status.
14. Shut down cleanly.
15. Recover from temporary failures.

---

77. Hardware Compatibility Testing

The Edge Agent shall be tested across multiple hardware configurations.

Example test matrix:

Device
CPU
RAM
Operating System
Camera
FPS
CPU Usage
Memory Usage
Inference Latency
Result

Test systems may include:

- Older laptops
- Mid-range laptops
- Modern laptops
- Integrated webcams
- External USB webcams

The final supported hardware specification shall be defined after benchmarking rather than assumed beforehand.

---

78. Multi-Computer Testing

Testing shall progressively increase the number of Edge Nodes.

Stage 1

1 Edge Node
+
1 Server

Stage 2

2 Edge Nodes
+

1 Server

Stage 3

4 Edge Nodes
+
1 Server

Stage 4

8 or more Edge Nodes
+
1 Server

The final number shall depend on available hardware.

---

79. Failure Testing

The following cases shall be tested.

Camera failure

Disconnect or disable the camera.

Expected behavior:

Application remains running.
Camera error is shown.

Central Server failure

Stop the server.

Expected behavior:

Local analysis continues.
Results are queued.

Network failure

Disconnect the network.

Expected behavior:

Local analysis continues.
Local feedback continues.
Data remains queued.

Server recovery

Restore the server.

Expected behavior:

Queued records synchronize automatically.

---

80. Privacy Testing

The project shall verify:

Raw video transmitted?
NO

Face image transmitted?
NO

Screenshot transmitted?
NO

Audio transmitted?
NO

Anonymized summary transmitted?
YES

Network inspection should be used during validation.

---

81. Performance Testing

Measure:

Edge

- FPS
- Inference latency
- CPU utilization
- RAM utilization
- Startup time
- Camera initialization time

Network

- Payload size
- Requests per telemetry window
- Data transferred per workstation
- Synchronization time

Central Server

- API response latency
- Database write time
- Aggregation time
- Report generation time
- Number of concurrent Edge Nodes

---

82. Accuracy Testing

The system shall be evaluated against reference cases.

Test:

- Landmark accuracy
- Angle consistency
- RULA correctness
- REBA correctness
- Risk classification
- Expression classification

Reference posture cases may be manually prepared or expert-labelled where available.

---

83. Unit Testing

Unit tests shall cover:

- Landmark transformations
- Angle calculation
- RULA rules
- REBA rules
- Risk classification
- Expression classification
- Temporal filtering
- Alert generation
- CSV generation
- Data validation
- Synchronization logic

---

84. Integration Testing

Test the complete Edge pipeline:

Camera
↓
OpenCV
↓
Pose / Face
↓
Landmarks
↓
Angles / Features
↓
Ergonomic / Expression Engine
↓
Temporal Engine
↓
Feedback
↓

SQLite
↓
Sync
↓
Central Server

---

85. End-to-End Testing

Perform complete deployment tests:

Clean Computer
↓
Install EdgeErgo
↓
Register Node
↓
Start Agent
↓
Camera Analysis
↓
Worker Overlay
↓
Local Storage
↓
Server Transmission
↓
Central Dashboard
↓
Report

---

86. Database Reliability

The system shall:

- Use transactions.
- Handle interrupted writes.
- Avoid duplicate uploads.
- Track synchronization status.
- Retry failed synchronization.

- Preserve queued records during temporary failures.

---

87. Duplicate Prevention

Every telemetry window should have a unique identifier.

Example:

NODE-004
+
WINDOW-20260914-1000

The Central Server should reject duplicate submissions rather than storing the same telemetry window multiple times.

---

88. API Requirements

Edge Registration

POST /api/v1/edge/register

Heartbeat

POST /api/v1/edge/heartbeat

Assessment Summary

POST /api/v1/edge/assessment

Configuration

GET /api/v1/edge/config

Dashboard Summary

GET /api/v1/dashboard/summary

Workstations

GET /api/v1/dashboard/workstations

Trends

GET /api/v1/dashboard/trends

Reports

GET /api/v1/reports/daily
GET /api/v1/reports/weekly
GET /api/v1/reports/monthly

CSV Export

GET /api/v1/export/csv

---

89. Data Validation

The Central Server shall validate:

- Device ID
- Session ID
- Timestamp
- Required fields
- Numeric ranges
- Risk categories
- Expression categories
- Telemetry window uniqueness

Invalid records shall not corrupt the database.

---

90. Logging

The Edge Agent shall maintain local logs for:

- Application startup
- Camera errors

- Model errors
- Sync errors
- Server connection errors
- Configuration changes
- Warning events

The Central Server shall maintain logs for:

- Node registration
- API errors
- Incoming telemetry
- Validation failures
- Database errors
- Report generation

Logs shall avoid storing unnecessary camera or facial data.

---

91. Recommended Project Structure

```
EdgeErgo/
│
├── edge_agent/
│   │
│   ├── main.py
│   │
│   ├── camera/
│   │   └── camera.py
│   │
│   ├── vision/
│   │   ├── preprocessing.py
│   │   ├── pose_engine.py
│   │   └── face_engine.py
│   │
│   ├── geometry/
│   │   └── angle_calculator.py
│   │
│   ├── ergonomics/
│   │   ├── rula.py
│   │   ├── reba.py
│   │   └── scoring.py
│   │
│   ├── expression/
```

```
│   │   └── classifier.py
│   │
│   ├── temporal/
│   │   └── analyzer.py
│   │
│   ├── feedback/
│   │   ├── alert_engine.py
│   │   ├── recommendations.py
│   │   └── scheduler.py
│   │
│   ├── overlay/
│   │   └── overlay.py
│   │
│   ├── storage/
│   │   └── local_db.py
│   │
│   ├── sync/
│   │   └── server_sync.py
│   │
│   ├── config/
│   │   └── config_manager.py
│   │
│   └── health/
│       └── heartbeat.py
│
├── central_server/
│   │
│   ├── main.py
│   │
│   ├── api/
│   │   ├── edge.py
│   │   ├── dashboard.py
│   │   └── reports.py
│   │
│   ├── aggregation/
│   │   └── aggregator.py
│   │
│   ├── analytics/
│   │   └── analytics.py
│   │
│   ├── database/
│   │   └── database.py
│   │
│   ├── reports/
```

```
│   │   └── report_generator.py
│   │
│   └── validation/
│       └── validators.py
│
├── dashboard/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── charts/
│   │   ├── services/
│   │   └── types/
│   │
│   └── package.json
│
├── installer/
│
├── tests/
│   ├── edge/
│   ├── server/
│   └── integration/
│
├── docs/
│   ├── architecture/
│   ├── deployment/
│   ├── api/
│   └── research/
│
├── README.md
└── requirements.txt
```

---

92. Development Phases

Phase 1 — Project Setup

- Git repository
- Python environment
- Project structure
- Dependency configuration
- Development conventions

Deliverable

Working development environment.

---

93. Phase 2 — Camera Module

Implement:

- OpenCV camera initialization
- Frame capture
- Frame resizing
- Camera selection
- Error handling

Deliverable

Working camera subsystem.

---

94. Phase 3 — Pose Engine

Implement:

- MediaPipe Pose
- Landmark extraction
- Confidence handling
- Skeleton visualization

Deliverable

Working local pose detection.

---

95. Phase 4 — Geometry Engine

Implement:

- Landmark coordinate processing
- Vector calculations
- Joint angles
- Smoothing

Deliverable

Reliable posture measurements.

---

96. Phase 5 — Ergonomic Engine

Implement:

- RULA
- REBA
- Scoring tables
- Risk classification

Deliverable

Validated ergonomic assessment.

---

97. Phase 6 — Facial Analysis

Implement:

- Face detection
- Facial landmarks/features
- Lightweight expression classification
- Confidence filtering

Deliverable

Local broad expression analysis.

---

98. Phase 7 — Temporal Engine

Implement:

- Monitoring interval
- Analysis duration
- Persistence detection
- Alert cooldown
- Session management

Deliverable

Scheduled and stable analysis.

---

99. Phase 8 — Worker Overlay

Implement:

- PySide6
- Transparent overlay
- Posture warnings
- Movement reminders
- Hydration reminders
- Status indicator

Deliverable

Worker-facing Edge UI.

---

100. Phase 9 — Local Storage

Implement:

- SQLite
- Sessions
- Assessments
- Events

- Sync queue

Deliverable

Offline-capable local data storage.

---

101. Phase 10 — Edge-to-Server Communication

Implement:

- Registration
- Heartbeat
- Telemetry upload
- Error handling
- Retry system
- Duplicate prevention

Deliverable

Working Edge-to-Central communication.

---

102. Phase 11 — Central Server

Implement:

- FastAPI
- Central SQLite
- Validation
- Device management
- Data ingestion

Deliverable

Working central backend.

---

103. Phase 12 — Multi-Node Testing

Connect:

NODE-001
NODE-002
NODE-003
NODE-004

to one server.

Deliverable

Working distributed Edge system.

---

104. Phase 13 — Collective Analytics

Implement:

- Average scores
- Risk distributions
- Workstation comparison
- Time trends
- Expression statistics
- Combined trends
- Break analysis

Deliverable

Collective analytics engine.

---

105. Phase 14 — Central Dashboard

Implement:

- Overview
- Active workstations

- Risk distribution
- Workstation table
- Trends
- Expression analysis
- Combined analysis

Deliverable

Administrator dashboard.

---

106. Phase 15 — CSV and Reporting

Implement:

- CSV export
- Daily reports
- Weekly reports
- Monthly reports
- Collective summaries

Deliverable

Reporting system.

---

107. Phase 16 — Packaging

Package:

EdgeErgo Agent
Central Server
Central Dashboard

into deployable components.

Deliverable

Installable system.

---

108. Phase 17 — Clean Computer Testing

Install EdgeErgo on computers without the development environment.

Verify:

- Installation
- Camera
- Registration
- Local AI
- Overlay
- SQLite
- Synchronization

Deliverable

Validated deployment package.

---

109. Phase 18 — Multi-Hardware Testing

Test on different hardware.

Measure:

- FPS
- CPU
- Memory
- Inference latency
- Stability

Deliverable

Hardware compatibility results.

---

110. Phase 19 — Failure and Privacy Testing

Test:

- Camera failure
- Server failure
- Network failure
- Low confidence
- Person leaves frame
- Multiple people
- Data synchronization
- Raw-video transmission

Deliverable

System validation report.

---

111. Phase 20 — Final Optimization

Optimize:

- Inference frequency
- Frame resolution
- Memory usage
- CPU usage
- Network traffic
- Database operations
- Server processing

Deliverable

Production-ready academic prototype.

---

112. Definition of Done — Edge Agent

The Edge Agent is considered complete when:

- Camera works reliably.
- Pose detection works.
- Facial analysis works.
- Required measurements are calculated.
- Ergonomic assessment is implemented.
- Temporal analysis works.
- Scheduled monitoring works.
- Worker overlay works.
- Local database works.
- Offline mode works.
- Synchronization works.
- Heartbeat works.
- Configuration works.
- Errors are handled.
- Application can be packaged.
- Application works on a clean computer.

---

113. Definition of Done — Central Server

The Central Server is considered complete when:

- Edge registration works.
- Multiple nodes can connect.
- Heartbeats are recorded.
- Telemetry is accepted.
- Duplicate records are prevented.
- Data is stored.
- Aggregation works.
- Trends work.
- CSV export works.
- Reports work.
- Dashboard APIs work.
- Server failure does not cause Edge data loss.

---

114. Definition of Done — Complete System

The entire system is considered complete when:

Multiple Computers
↓
Independent Local AI
↓
Local Feedback
↓
Local Storage
↓
Anonymized Summaries
↓
Central Server
↓
Collective Analysis
↓
Dashboard
↓
CSV
↓
Detailed Report

works from end to end.

---

115. Key Performance Indicators

Edge Performance

- Pose inference FPS
- Face inference FPS
- CPU utilization
- Memory utilization
- End-to-end latency
- Startup time

Network Performance

- Average payload size
- Data transferred per telemetry window
- Synchronization time
- Number of API requests

Central Performance

- API latency
- Database write latency
- Aggregation latency
- Dashboard response time
- Report generation time

Assessment

- Landmark reliability
- Angle consistency
- RULA correctness
- REBA correctness
- Risk classification consistency
- Expression classification consistency

User Experience

- Feedback latency
- Alert usefulness
- Overlay usability
- Dashboard clarity
- Report clarity

Privacy

- Raw-video transmission
- Face-image transmission
- Unnecessary data transmission
- Local retention
- Server-side stored data

---

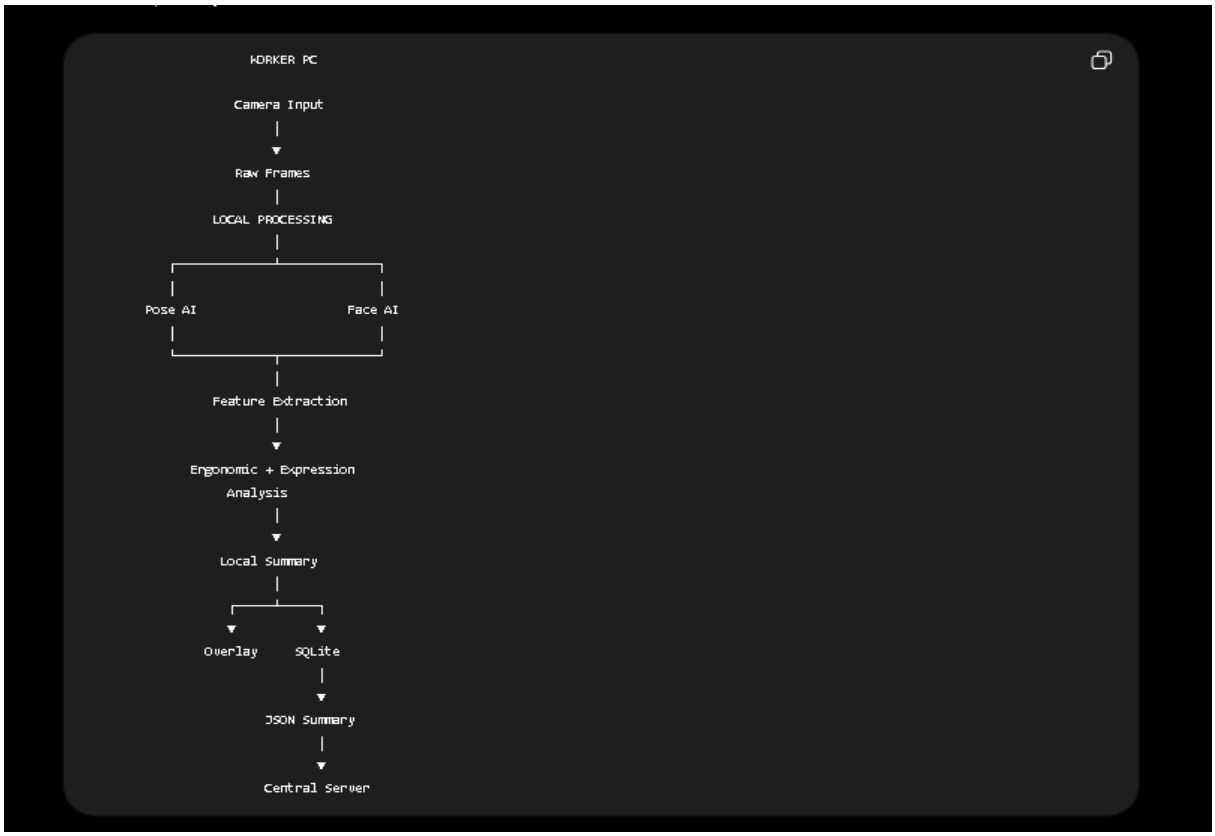
116. Risks and Mitigation

Risk	Impact	Mitigation
Poor camera position	Incorrect measurements	Camera setup guidance
Low lighting	Reduced detection quality	Confidence checks
Body occlusion	Missing landmarks	Visibility filtering
Multiple people	Incorrect subject	Single-worker constraint
Temporary pose failure	False warnings	Temporal filtering
High CPU	Poor FPS	Frame skipping and reduced resolution
Network failure	Data synchronization delay	Local queue
Server failure	Temporary central unavailability	Local Edge operation
Incorrect ergonomic mapping	Invalid scores	Validate scoring engine
Excessive alerts	User annoyance	Cooldown and persistence
Expression misclassification	Incorrect aggregate data	Confidence filtering and validation
Data duplication	Invalid statistics	Unique telemetry IDs
Privacy leakage	Major security issue	Local-only media processing
Installation failure	Deployment difficulty	Packaged installer and clean-PC testing

---

117. Privacy Architecture

The final privacy architecture shall be:

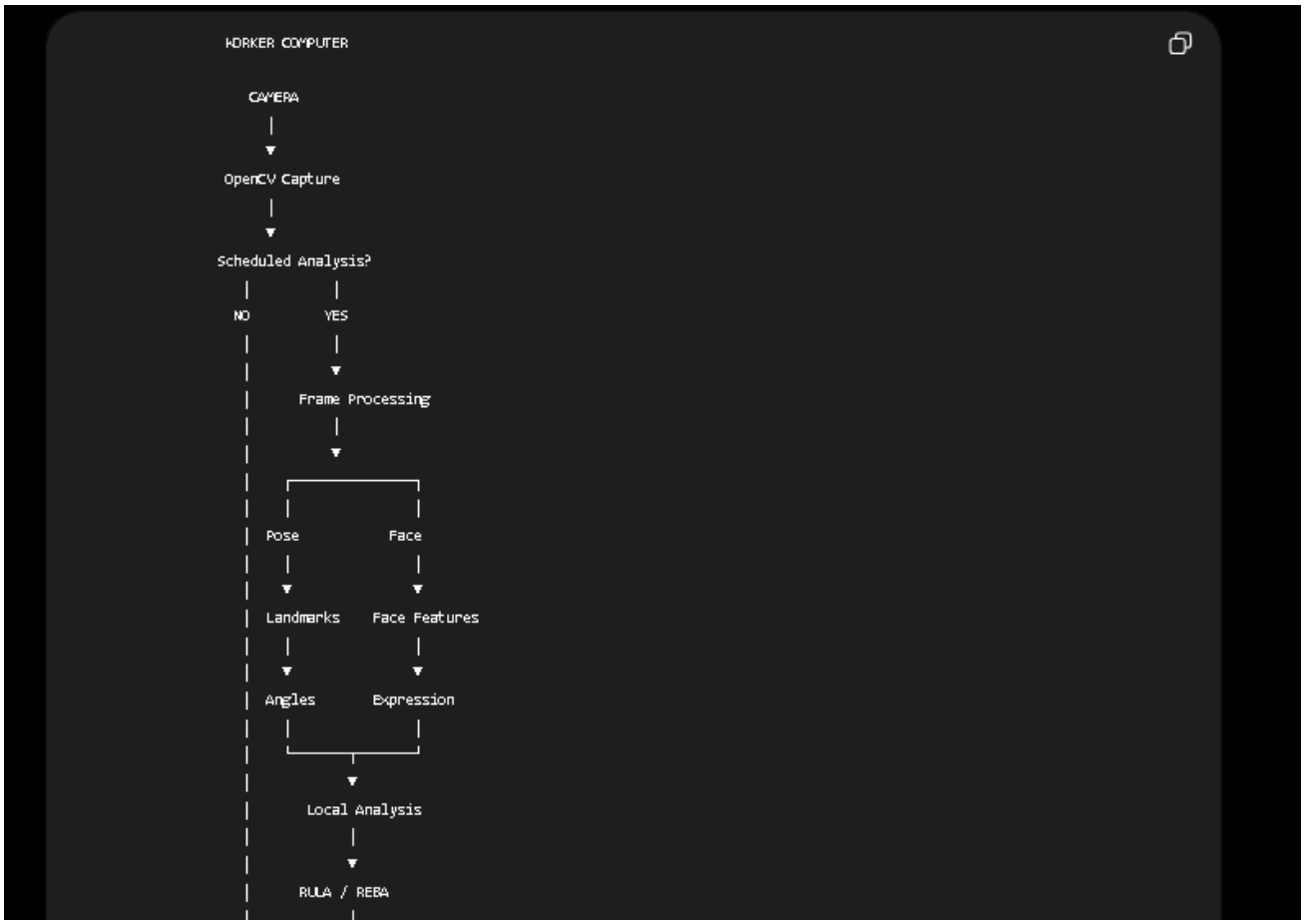


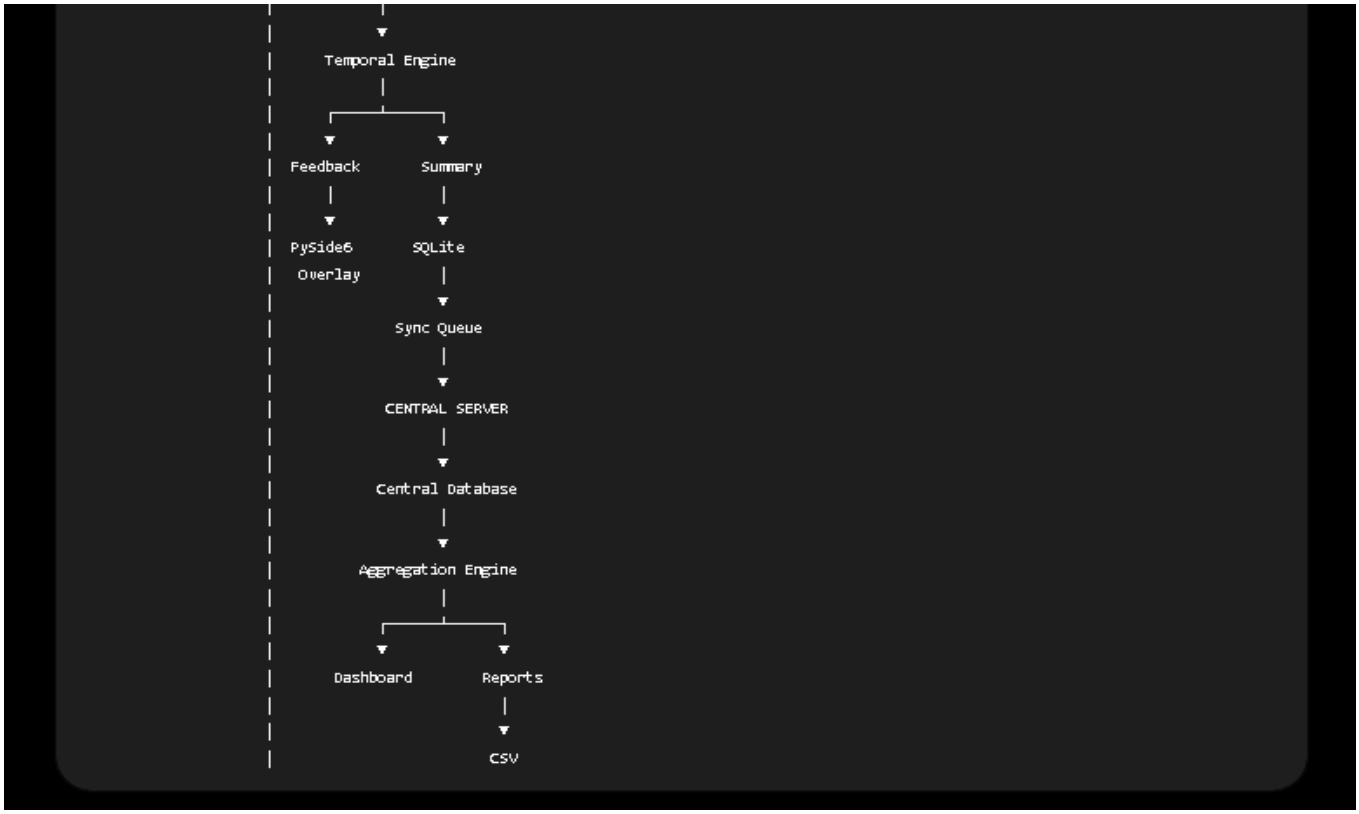
No raw camera stream is required by the Central Server.

---

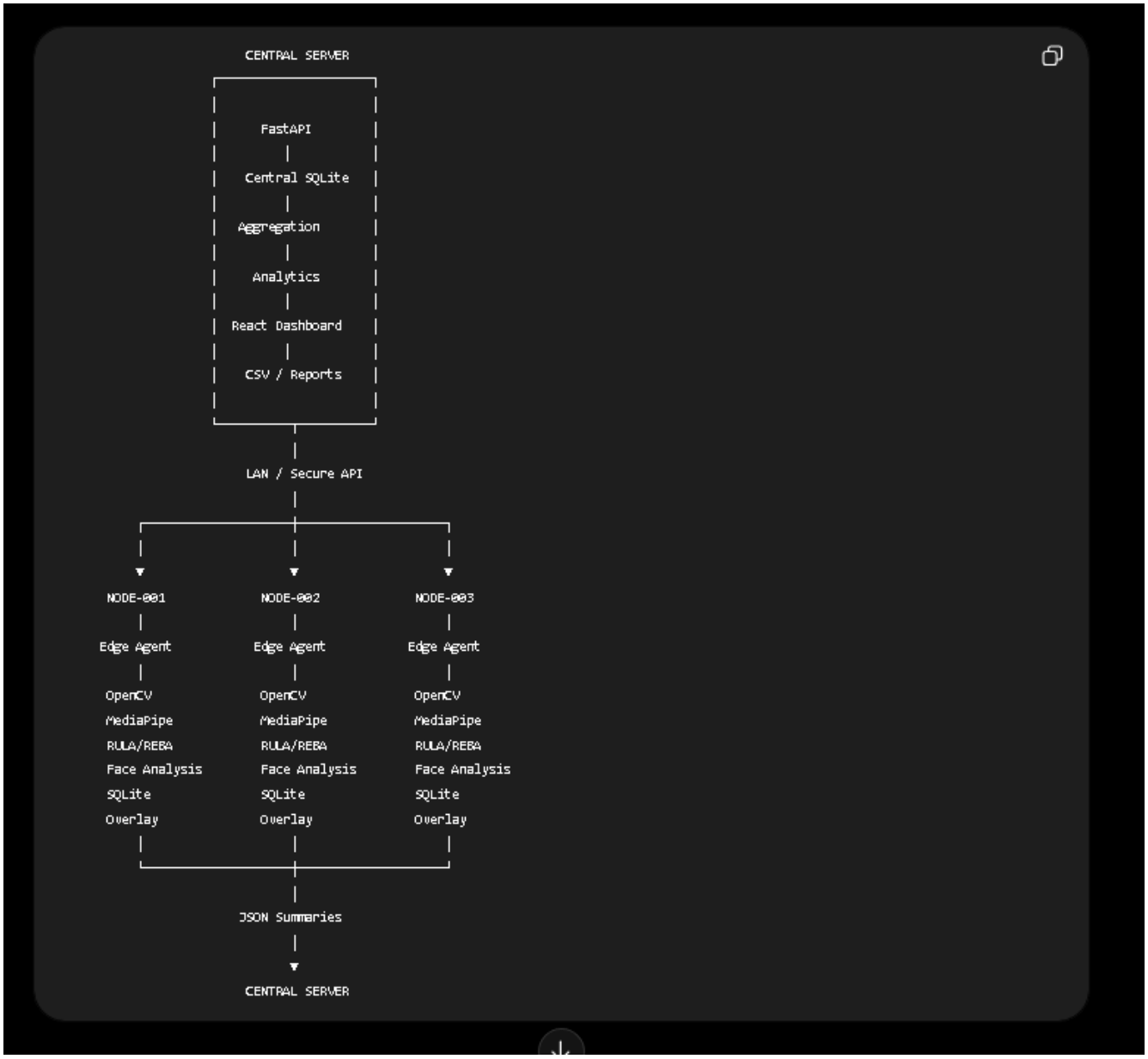
118. Final Product Flow

The final EdgeErgo flow shall be:




Temporal Engine
Feedback
Summary
PySide6
Overlay
SQLite
Sync Queue
CENTRAL SERVER
Central Database
Aggregation Engine
Dashboard
Reports
CSV

119. Final Multi-Workstation Architecture



---

120. Final Product Definition

EdgeErgo is a **distributed privacy-preserving Edge AI system** in which:

Each worker computer independently performs:

Camera Capture
→ Pose Detection
→ Facial Analysis
→ Angle Calculation
→ Ergonomic Assessment
→ Temporal Analysis
→ Local Feedback
→ Local Storage

while the **Central Server performs:**

Node Management
→ Data Collection
→ Aggregation
→ Collective Analysis
→ Dashboard
→ CSV
→ Reports

The worker's computer remains responsible for the computationally intensive computer-vision processing.

The Central Server does not need to process raw camera streams.

---

121. Central Technical Objective

The central technical objective of EdgeErgo is:

> **To demonstrate a lightweight distributed Edge AI architecture where multiple worker computers independently analyze ergonomic posture and broad facial-expression states, provide immediate local feedback, and transmit only anonymized analytical summaries to a central system for collective workplace analysis and reporting.**

The complete technical pipeline is:

MULTIPLE CAMERAS
↓
LOCAL EDGE AI
↓
POSE + FACE FEATURES
↓

ERGONOMIC + EXPRESSION ANALYSIS
↓
TEMPORAL ANALYSIS
↓
LOCAL WORKER FEEDBACK
↓
LOCAL AGGREGATION
↓
ANONYMIZED JSON
↓
CENTRAL SERVER
↓
COLLECTIVE AGGREGATION
↓
ANALYTICS
↓
DASHBOARD
↓
CSV
↓
DETAILED REPORT

---

122. Final Success Criteria

EdgeErgo will be successful if the final prototype demonstrates all of the following:

1. At least multiple computers can operate as independent Edge Nodes.
2. Each Edge Node performs posture analysis locally.
3. Facial-expression analysis is performed locally.
4. RULA/REBA assessment is performed locally.
5. Analysis occurs according to configurable time intervals.
6. Persistent posture problems trigger local feedback.
7. The worker can see the system status and warnings through the overlay.
8. Movement and hydration reminders can be displayed locally.
9. Raw video does not need to leave the workstation.
10. Each Edge Node stores its data locally.
11. Local analysis continues when the server is unavailable.
12. Unsynchronized data is queued locally.
13. Queued data synchronizes after server recovery.
14. Multiple Edge Nodes can communicate with one Central Server.
15. The Central Server maintains node health information.
16. The Central Server aggregates telemetry.

17. Collective ergonomic trends can be visualized.
18. Aggregate facial-expression statistics can be visualized without sending face images.
19. CSV export works.
20. Detailed reports can be generated.
21. The Edge Agent can be installed on clean computers.
22. The application works across multiple tested hardware configurations.
23. Performance is benchmarked.
24. Privacy behavior is tested.
25. The complete system works from camera input to final report.

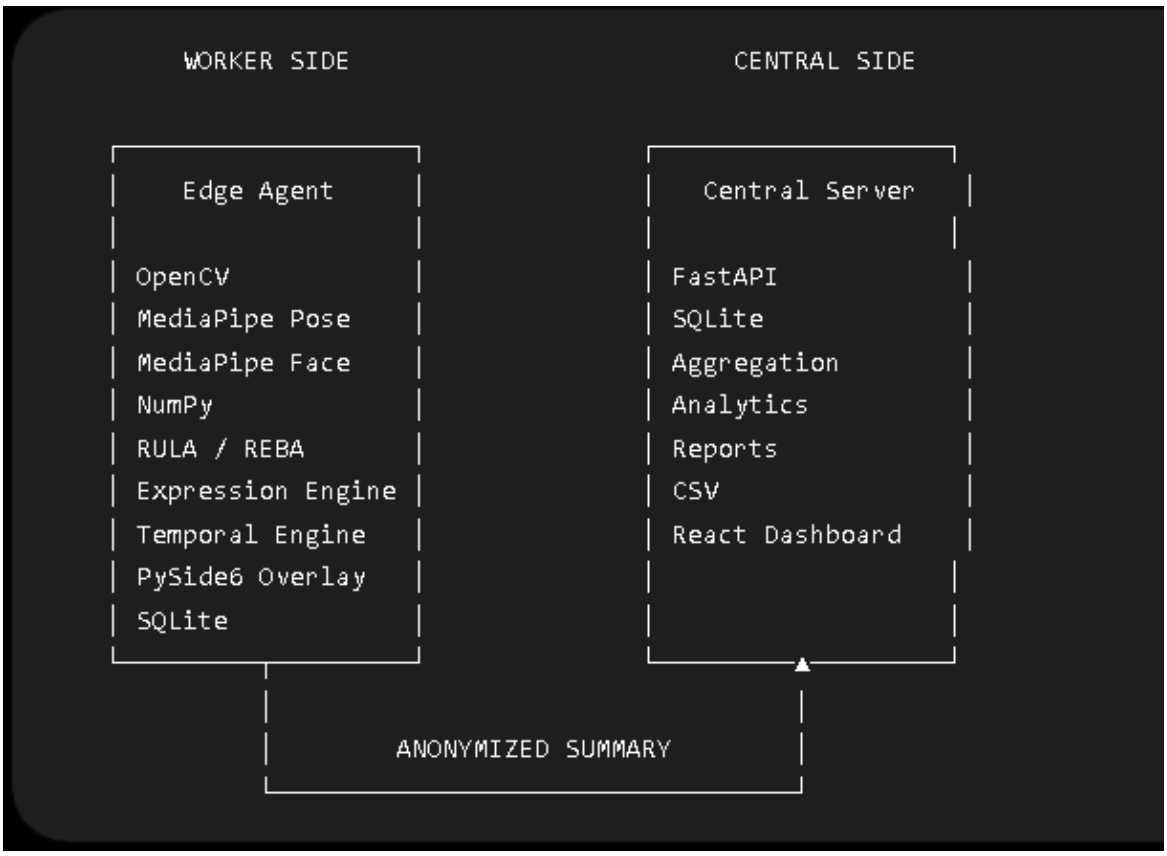
---

123. Final Architecture Statement

EdgeErgo is not a centralized camera-monitoring application.

It is a **distributed Edge AI system**.

The fundamental architecture is:



The worker receives **local intelligence**, while the central server receives **collective intelligence**.

This architecture shall remain the foundation for subsequent implementation decisions.